

Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey.

Inteligencia Artificial Avanzada para la Ciencia de Datos
“Reto CAETEC | Reto Datos”

Profesor:
Ismael Solís Moreno

Jorge Armando Garzón Varela | A01707102
Santiago Alducin Villaseñor | A01707122
Sebastián Pérez García | A01707346
Roberto Serna Niño | A01709577
Kamila Jeannette Martinez Ibarra | A01711833

Fecha de entrega: 23 de octubre de 2026

Introducción

El presente documento forma parte de nuestro proyecto de ciencia de datos, desarrollado en colaboración con el CAETEC. El proyecto tiene como objetivo analizar los registros del sistema de ordeño voluntario (VMS) para identificar patrones temporales y condiciones asociadas con la formación de colas de vacas. A través de este análisis, buscamos comprender las circunstancias en las que se presentan estas colas y generar información valiosa que contribuya a la toma de decisiones y a la mejora en la eficiencia del proceso de ordeño.

Para el desarrollo del proyecto se utiliza la metodología CRISP-DM, que permite organizar las actividades en distintas etapas de comprensión, preparación, modelado y evaluación de los datos. En este documento se describen las herramientas y tecnologías seleccionadas, los modelos de almacenamiento y los flujos de datos propuestos, así como una prueba de concepto utilizando PySpark. Además, se presenta una posible arquitectura de producción alternativa basada en AWS y una reflexión sobre la necesidad de utilizar tecnologías de Big Data considerando el alcance, los requerimientos, y las características actuales del proyecto.

Para consultar información adicional sobre el contexto total del proyecto, su estado actual de desarrollo y los avances realizados en cada etapa, se puede acceder al repositorio de GitHub del equipo, el cual está disponible en: https://github.com/kamimartinez/VMS-prediction/.

1. Herramientas y Tecnologías

A continuación, se describen las herramientas y tecnologías a utilizar durante las fases de CRISP-DM, comparando el entorno de desarrollo frente al entorno de producción escalable.

La selección de herramientas y tecnologías se realizó considerando las necesidades de procesamiento, almacenamiento, modelado y evaluación del proyecto, así como los recursos computacionales disponibles y los datos proporcionados. Para ello, se distinguen dos escenarios: un entorno de desarrollo académico, orientado a la experimentación y validación de resultados, y un entorno de producción hipotético, diseñado para soportar mayores volúmenes de información, procesamiento distribuido y acceso mediante una aplicación web.

Esta comparación permite justificar las decisiones tecnológicas adoptadas durante el proyecto e identificar posibles alternativas si se requiriera una implementación a mayor escala. En la siguiente tabla se presentan las tecnologías propuestas para cada fase de CRISP-DM:

Fase de CRISP-DM	Tecnologías en Desarrollo	Tecnologías en Producción
Business Understanding	Web of Science, Excel, Google Drive	Web of Science, Excel, Google Drive
Data Understanding	Pandas, NumPy, Matplotlib, Seaborn, Google Colab	PySpark, AWS EMR, AWS S3
Data Preparation	Pandas, NumPy, Google Colab	PySpark, AWS EMR
Data Modeling	TensorFlow, Keras, Pandas, Google Colab, Scikit-Learn	PySpark MLlib, Spark ML, Docker / Kubernetes / AWS EMR
Evaluation	Matplotlib, Seaborn, Google Colab	Tableau, Chart.js
Deployment	AWS Learner Lab / Prototipo Local	AWS (API Gateway, EC2, S3, Load Balancer, NAT, EMR, DocumentDB)

Como se observa en la tabla anterior, las principales diferencias entre ambos entornos se encuentran en las fases que requieren mayor capacidad de procesamiento y almacenamiento de información. Mientras que el entorno de desarrollo prioriza herramientas accesibles y suficientes para el volumen actual de los datos, el entorno de producción contempla servicios en la nube y tecnologías de cómputo distribuido capaces de trabajar con un volumen mayor.

A continuación, se justifica la selección de estas herramientas y tecnologías de acuerdo con las actividades y necesidades específicas de cada una de las etapas de CRISP-DM:

Business Understanding (Más información)

Para Business understanding las tecnologías que se utilizan no serían muy diferentes entre desarrollo y producción. Debido a que esta etapa consiste en entender el negocio, mencionamos Web of Science como fuente de datos para la investigación previa y comprender el contexto, Excel para la visualización rápida de los datos y un análisis simple de la información proporcionada, y Drive ya que es la herramienta donde estaremos trabajando la documentación y el medio por el cual hemos recibido los archivos de datos.

Data understanding (Más información)

El reto lo vamos a trabajar utilizando Python, y las librerías base de Pandas y Numpy. Esto tomando en consideración el tamaño de los datos y el ambiente de desarrollo que estamos usando en el proyecto. El conjunto de datos es alrededor de 300 MB dentro de un objeto de pandas, una cantidad más que razonable para trabajarla en una sola unidad de memoria en una computadora promedio. Además, este tamaño no representa ningún problema en nuestro caso dado que estaremos trabajando en Google-Colab, que nos brinda más poder de cómputo. Por este motivo, los beneficios de herramientas de big-data como Spark y su habilidad de usar múltiples computadoras o el tiempo de procesamiento no son necesarios para desarrollo.

En cambio, en un ambiente de producción donde la cantidad de datos sea exponencialmente mayor, Pandas y Numpy se quedarían cortos. Alternativamente a estas librerías de Python se propone utilizar Amazon EMR (Elastic MapReduce) el cual nos permitiría administrar y ejecutar procesos de Spark para la instanciación de múltiples contenedores que ejecuten el procesamiento de datos de manera distribuida, además de Amazon S3 para almacenar de manera centralizada los conjuntos de datos, facilitando su acceso y procesamiento desde EMR.

Data Preparation

Posterior a esto, para Data Preparation seguimos utilizando las mismas tecnologías en la limpieza de los datos, la creación y selección de columnas necesarias, y la preparación de los datos para el entrenamiento del modelo. En un ambiente local de desarrollo, Pandas y Numpy siguen siendo suficientes en nuestro caso, sin embargo, en un ambiente de producción, igual recurriríamos al uso de EMR y Spark para acelerar el proceso mediante cómputo distribuido.

Modeling

Para la etapa de modelado utilizaremos la librería de Tensorflow con Keras ya que tiene múltiples funciones y herramientas integradas, las cuales permiten construir algoritmos y modelos de aprendizaje como regresiones y redes neuronales. Dentro de esta etapa, los datos ya procesados se cargan con Pandas y se separan en las variables de entrada y la variable objetivo. Posteriormente, se utilizará Scikit-Learn para realizar la partición de los datos mediante validación cruzada k-fold cross-validation y el escalamiento de las variables numéricas. Para evitar la fuga de información, el escalador se ajustará únicamente con los datos de entrenamiento de cada fold y posteriormente se aplicará a los datos de validación correspondientes. Después, los datos se convierten en arreglos de NumPy para alimentar al modelo por lotes durante el entrenamiento. Para finalizar, la salida de este proceso se genera como un conjunto de pesos, en otras palabras el modelo entrenado, el cual se guarda en formato .keras junto con las métricas obtenidas en cada fold para su posterior evaluación.

Por otro lado, en un ambiente de producción con mayor volumen de datos podemos utilizar PySpark - MLlib y ML para aprovechar el procesamiento distribuido, haciendo que el entrenamiento del modelo sea mucho más rápido y eficiente. Este sería un enfoque diferente en el entrenamiento, mas la entrada y la salida serían las mismas que en el modelo regular.

Evaluation

Finalmente para la evaluación estaríamos utilizando las librerías Matplotlib y Seaborn para la visualización del proceso de entrenamiento y los resultados. Gracias a la utilización de estas librerías tendremos la capacidad de graficar las métricas de desempeño de los modelos en cada iteración, asimismo ayudaría a la comparación de resultados y a la detección de posibles problemas, como es el caso del overfitting. No solo eso, sino que también ayudaría a visualizar patrones temporales de nuestra información, como el caso de la aparición de colas.

Sin embargo, en el caso de que el proyecto tuviera una mayor cantidad de información a tratar, una herramienta como Tableau podría presentar la información de mejor manera, sobre todo para los usuarios no técnicos. Por otro lado, en caso de utilizar una interfaz web entonces se podrían utilizar otro tipo de librerías del ecosistema de JavaScript como es el caso de Chart.js, la cual puede construir gráficas interactivas directamente en la aplicación.

Deploy

Para el objetivo de la clase se puede desplegar el modelo mediante una aplicación web utilizando servicios de AWS, sin embargo, de acuerdo al alcance actual del proyecto, no es necesario un despliegue ya que no existirán usuarios reales, solo una demostración en clase.

En cambio, si se tratara de un despliegue a producción para un uso industrial, si sería necesario un despliegue robusto. En la siguiente sección se muestra mejor la interacción de los componentes, pero este consistiría en una capa pública que permita la interacción de usuarios externos con el modelo, una capa interna que exponga esta interfaz web, un bucket Amazon S3 que permita almacenar los datos en crudo y los modelos generados, Amazon EMR para los trabajos de entrenamiento y una base de datos de documentos, como podrían ser Amazon DynamoDB o DocumentDB para almacenar metadata de los modelos generados.

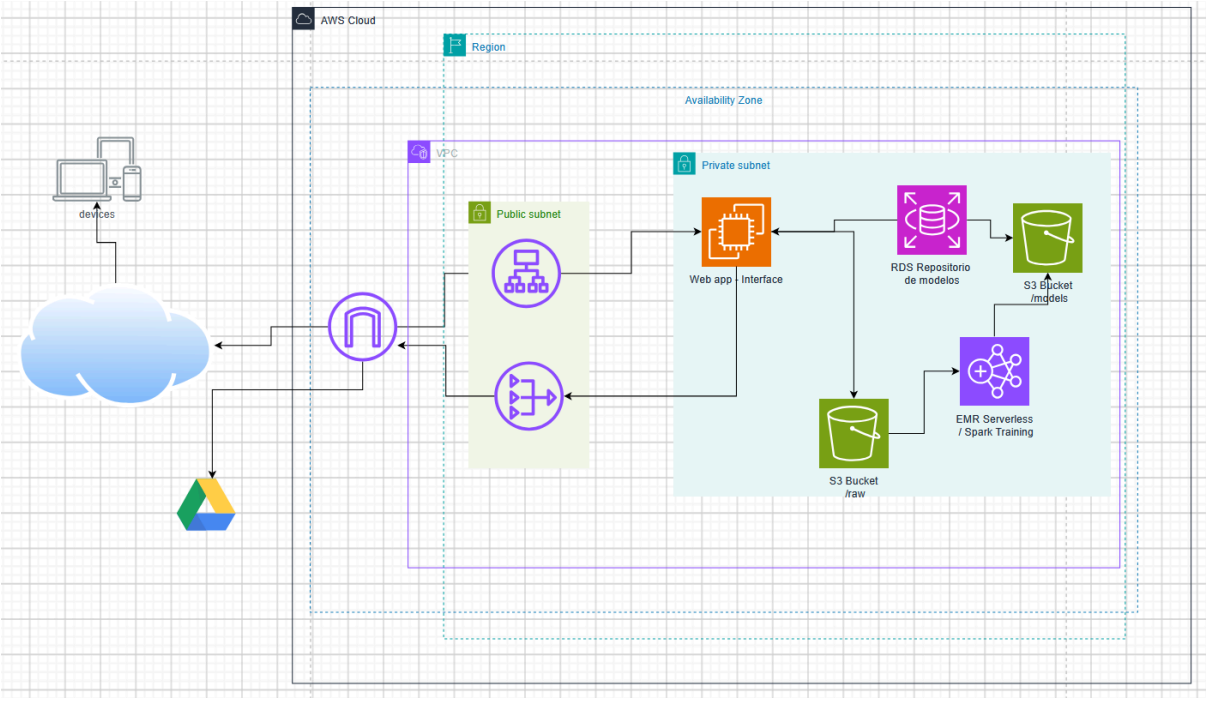
En conjunto, las tecnologías seleccionadas permiten cubrir las actividades correspondientes a las distintas fases de CRISP-DM, considerando tanto los objetivos académicos del proyecto como sus posibles necesidades futuras de producción. El entorno de desarrollo propuesto resulta adecuado para el volumen actual de datos y los recursos disponibles, sin requerir de una infraestructura distribuida ni servicios que agreguen una mayor complejidad innecesaria.

Por otra parte, las tecnologías consideradas para un entorno de producción representan alternativas de escalabilidad que podrían implementarse en caso de aumentar el volumen de información, la frecuencia de procesamiento o la cantidad de usuarios. De esta manera, se establece una distinción entre las herramientas necesarias para cumplir los objetivos actuales del proyecto y aquellas que podrían incorporarse en una implementación de mayor escala.

2. Modelo de Almacenamiento de Datos

Para una posible implementación del proyecto en un entorno de producción, se propone una arquitectura basada en servicios de Amazon Web Services (AWS), orientada al almacenamiento centralizado de información, el procesamiento distribuido de datos y la administración de los modelos generados. Esta arquitectura contempla la integración de los diferentes componentes necesarios para la ingesta, preparación, entrenamiento y almacenamiento de los modelos, así como su posterior uso mediante una aplicación web.

La propuesta considera la separación entre los servicios de almacenamiento, procesamiento y acceso a la aplicación, con el objetivo de facilitar la escalabilidad, la seguridad y la administración de recursos. A continuación, se presenta el diagrama de arquitectura propuesto:



La arquitectura propuesta utiliza distintos servicios proporcionados por AWS para distribuir las responsabilidades del sistema. Estos componentes están pensados y organizados de acuerdo con sus funciones de comunicación, procesamiento y almacenamiento de datos.

Capa de Red y Acceso a la Aplicación

Para gestionar el acceso de los usuarios externos, se propone utilizar Amazon API Gateway como punto de entrada a los servicios de la aplicación. Las solicitudes se dirigirían hacia un Load Balancer, encargado de distribuir el tráfico entre las instancias EC2 que ejecuten la aplicación web como interfaz con la creación y ejecución de modelos. Estos componentes permitirían administrar las peticiones de los usuarios y facilitar la escalabilidad del sistema.

Las instancias EC2 se ubicarían dentro de una subred privada de una Virtual Private Cloud (VPC), con el objetivo de restringir su exposición directa a Internet. Adicionalmente, se utilizaría un NAT para permitir conexiones salientes desde las instancias privadas hacia servicios externos, como Google Drive, sin necesidad de asignarles direcciones IP públicas.

Capa de Cómputo y Procesamiento

Amazon EC2 se utilizaría para ejecutar la lógica de negocio de la aplicación web, incluyendo la administración de solicitudes, la integración con Google Drive y la coordinación de los procesos de entrenamiento y ejecución de modelos. La aplicación permitiría obtener los archivos de datos proporcionados mediante Google Drive y almacenarlos posteriormente en Amazon S3. A partir de estos archivos, se podrían iniciar trabajos de carga, procesamiento y entrenamiento utilizando Amazon EMR (Elastic MapReduce), servicio encargado de administrar la infraestructura necesaria para ejecutar Spark de manera distribuida.

Mediante Spark, las tareas de preparación y procesamiento de datos podrían distribuirse entre diferentes nodos del clúster, aprovechando el procesamiento paralelo para trabajar con volúmenes de información mayores. Una vez finalizado el entrenamiento, los modelos generados se almacenarían en Amazon S3 para su consulta y utilización por la aplicación.

Almacenamiento de Datos y Modelos

Amazon S3 funcionaría como el repositorio centralizado de los archivos utilizados y generados durante las diferentes etapas de CRISP-DM en nuestro proyecto. Su implementación permitiría separar el almacenamiento de los recursos computacionales, facilitando el acceso a la información desde los servicios encargados del procesamiento. Se propone organizar el almacenamiento mediante dos prefijos principales dentro de un bucket:

- **/raw:** destinado a los archivos originales obtenidos desde Google Drive, los cuales servirían como entrada para las tareas de procesamiento y preparación de datos.
- **/models:** destinado a almacenar los modelos entrenados y sus archivos asociados, permitiendo conservar diferentes versiones para su comparación y utilización.

Durante el procesamiento, Spark accedería a los archivos almacenados en S3 y distribuiría las operaciones entre los distintos nodos administrados por Amazon EMR. Posteriormente, los resultados y modelos generados podrían escribirse nuevamente en S3, manteniendo un almacenamiento centralizado independiente de los nodos utilizados para el procesamiento.

Almacenamiento de Metadatos

Para administrar la información relacionada con todos los modelos generados, se propone utilizar Amazon Relational Database Service (RDS), mediante una base de datos relacional que permita mantener una estructura definida para el registro y consulta de metadatos.

Cada modelo entrenado contaría con un registro que almacenaría información como su nombre, versión, fecha y hora de creación, así como la ubicación del archivo correspondiente en Amazon S3. De esta manera, la aplicación podría consultar los modelos disponibles y sus archivos sin necesidad de almacenar directamente los modelos dentro de la base de datos.

Se seleccionó una base de datos relacional debido a las ventajas que ofrece una estructura definida mediante esquemas y a la facilidad para mantener registros consistentes. Sin embargo, también se podrían evaluar alternativas como Amazon DynamoDB o DocumentDB.

La arquitectura que estamos proponiendo nos permite organizar el almacenamiento de datos, el procesamiento distribuido y la administración de los modelos mediante servicios como Amazon S3, Amazon EMR, Amazon EC2 y Amazon RDS. Aunque esta infraestructura no es necesaria para el alcance actual de nuestro proyecto, nos da una alternativa para una implementación en producción, permitiendo aumentar la capacidad de procesamiento y almacenamiento en caso de requerirse según necesidades futuras que podría tener el proyecto.

3. Flujo de Datos

En esta sección se describe el flujo de datos del proyecto, desde la obtención de los datos del CAETEC hasta el procesamiento, entrenamiento y evaluación de los modelos. Al igual que en las secciones anteriores, se consideran dos escenarios: el entorno de desarrollo académico, utilizando servicios de Google, y un entorno de producción basado en los servicios de AWS.

El objetivo es identificar cómo se mueven los datos entre las distintas etapas, así como las diferencias en el almacenamiento y procesamiento de los datos entre ambos entornos.

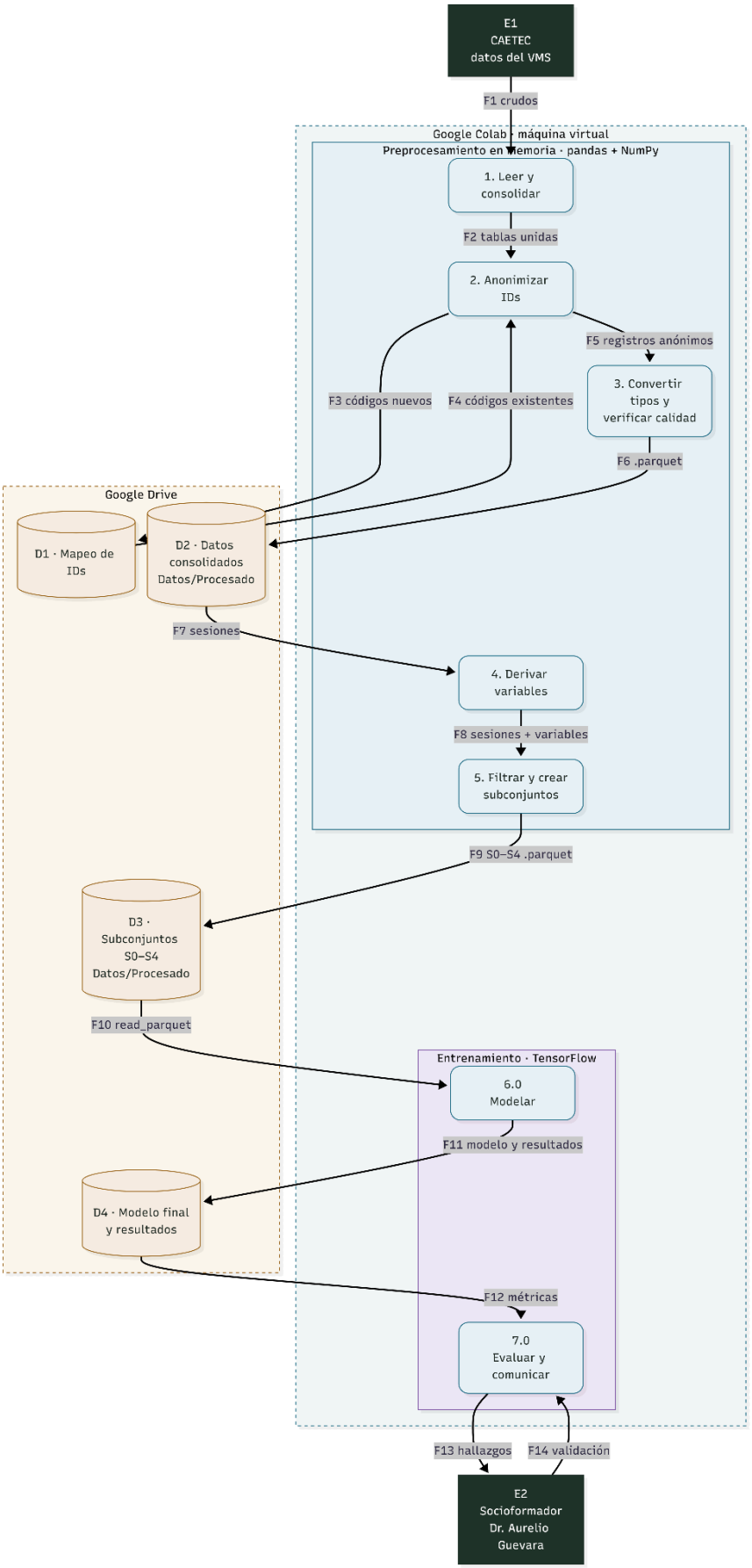
Flujo de Datos en Desarrollo

En el entorno de desarrollo, los datos originales proporcionados por CAETEC se encuentran almacenados en Google Drive, dentro de una carpeta llamada Datos/Crudo. Estos archivos contienen información de producción y consumo de las vacas durante el periodo de 2022 a 2026. Para trabajar con ellos, utilizamos Google Colab, que permite cargar los archivos en DataFrames de Pandas y procesarlos en memoria dentro del entorno de ejecución de Python.

Posteriormente, usando Pandas y NumPy, realizamos la limpieza, transformación y preparación de los datos. Este proceso incluye la consolidación de los archivos, la anonimización de los identificadores de las vacas, la verificación de la calidad de los registros y la creación de nuevas variables relacionadas con los ordeños y la formación de colas. Los datos procesados se almacenan nuevamente en Google Drive, dentro de otra carpeta llamada Datos/Procesada, en archivos .parquet para facilitar su recuperación en las siguientes etapas.

Una vez preparados los datos, se utilizan los subconjuntos generados para entrenar el modelo mediante TensorFlow y Keras dentro de Google Colab. Finalmente, el modelo entrenado y sus resultados se almacenan en Google Drive para su evaluación. Las métricas permiten analizar el desempeño del modelo y presentar los hallazgos al socio para su validación.

A continuación, se presenta el diagrama de flujo de datos del entorno de desarrollo, donde se muestran los distintos procesos, componentes y las interacciones entre los mismos:


E1
CAETEC
datos del VMS
F1 crudos
Google Colab · máquina virtual
Preprocesamiento en memoria · pandas + NumPy
1. Leer y consolidar
F2 tablas unidas
2. Anonimizar IDs
F5 registros anónimos
3. Convertir tipos y verificar calidad
F3 códigos nuevos
F4 códigos existentes
F6 .parquet
Google Drive
D1 · Mapeo de IDs
D2 · Datos consolidados
Datos/Procesado
F7 sesiones
4. Derivar variables
F8 sesiones + variables
5. Filtrar y crear subconjuntos
F9 S0–S4 .parquet
D3 · Subconjuntos S0–S4
Datos/Procesado
F10 read_parquet
Entrenamiento · TensorFlow
6.0 Modelar
F11 modelo y resultados
D4 · Modelo final y resultados
F12 métricas
7.0 Evaluar y comunicar
F13 hallazgos
F14 validación
E2
Socioformador
Dr. Aurelio Guevara

El diagrama contempla siete procesos principales, desde la lectura y consolidación de los registros hasta la evaluación y comunicación de resultados. La siguiente tabla describe lo que ocurre en cada proceso del diagrama de flujo de datos. Los procesos 1.0 a 5.0 forman el preprocesamiento con Pandas y NumPy dentro de Google Colab, y los procesos 6.0 y 7.0 corresponden al entrenamiento del modelo con TensorFlow/Keras y a su evaluación.

ID	Contenido
1.0	Leer y consolidar: Google Colab carga los archivos crudos de Drive (Datos/Crudo) en DataFrames de Pandas, asigna los nombres de columna, toma el identificador de cada vaca del nombre del archivo, une los registros y los ordena por hora de inicio.
2.0	Anonimizar IDs: reemplaza el identificador original de cada vaca por un código VACA_### y guarda la correspondencia en D1 (Mapeo de IDs).
3.0	Convertir tipos y verificar calidad: convierte la duración y el intervalo de ordeño a minutos, extrae las variables de fecha y revisa nulos, duplicados, traslapes por robot y valores atípicos.
4.0	Derivar variables: calcula el tiempo libre entre ordeños por robot, la posible cola (≤ 2 min), la ocupación del robot, los secados (≥ 40 días), los días en leche, la etapa de lactancia, los minutos desde el alimento y los ordeños forzados.
5.0	Filtrar y crear subconjuntos: aplica los filtros de la base limpia (duración de 1 a 30 min, producción mayor a 0 e intervalo mayor a 0), identifica los episodios de cola y construye los subconjuntos S0 a S4.
6.0	Modelar: lee los subconjuntos .parquet desde Drive y entrena el modelo con TensorFlow en Google Colab.
7.0	Evaluar y comunicar: evalúa el modelo con sus métricas, contrasta los resultados con los criterios de éxito y presenta los hallazgos al socioformador.

Además de los procesos, se identifican los componentes y flujos de información entre las diferentes etapas. Estos incluyen los archivos originales, los identificadores anonimizados, los subconjuntos procesados, los modelos entrenados y los resultados. La siguiente tabla describe cada flujo de datos: qué información se mueve, de dónde sale y a dónde llega.

ID	Contenido
F1	Datos crudos: CAETEC (E1) entrega los datos del VMS almacenados en Google Drive (Datos/Crudo): producción y consumo de las vacas de 2022 a 2026. Google Colab los carga en DataFrames de Pandas para el proceso 1.0.
F2	Tablas unidas: el proceso 1.0 envía al proceso 2.0 las tablas consolidadas, con los identificadores (id_vaca) y el archivo de origen (origen_archivo).
F3	Códigos nuevos: el proceso 2.0 guarda en D1 cada identificador de vaca que aparece por primera vez, junto con el código VACA_### que se le asignó.
F4	Códigos existentes: D1 devuelve al proceso 2.0 los códigos ya asignados, para que una misma vaca conserve siempre el mismo código.
F5	Registros anónimos: el proceso 2.0 envía al proceso 3.0 los registros con el identificador original reemplazado por su código.
F6	.parquet: el proceso 3.0 guarda en D2 las tablas consolidadas, con tipos convertidos y sin filtrar, como archivos .parquet.
F7	Sesiones: D2 entrega al proceso 4.0 las sesiones de ordeño (hora de inicio, duración, robot, producción e incidencias).
F8	Sesiones + variables: el proceso 4.0 envía al proceso 5.0 las sesiones con las variables derivadas: tiempo libre entre ordeños, posible cola, días en leche, etapa de lactancia, minutos desde el alimento, fin de semana y vacas activas.
F9	S0–S4 .parquet: el proceso 5.0 guarda en D3 los subconjuntos como archivos .parquet: S0 base limpia, S1 ventanas de 30 minutos por robot, S2 sesiones con etapa de lactancia, S3 episodios de cola y S4 vaca-día.
F10	read_parquet: D3 entrega al proceso 6.0 los subconjuntos, leídos con pd.read_parquet, para entrenar el modelo.
F11	Modelo y resultados: el proceso 6.0 guarda en D4 (Google Drive) el modelo final entrenado y sus resultados.
F12	Métricas: D4 entrega al proceso 7.0 las métricas del modelo para evaluarlo y desplegar los resultados.
F13	Hallazgos: el proceso 7.0 presenta al socioformador (E2) los patrones y hallazgos sobre cuándo y en qué condiciones se forman las colas.
F14	Validación: el socio (E2) regresa al proceso 7.0 su validación de los hallazgos.

Flujo de Datos en Producción

Para el entorno de producción, se propone un flujo de datos basado en el modelo de almacenamiento de datos que usa los servicios de AWS descritos en la sección anterior. A diferencia del entorno de desarrollo, este escenario contempla el uso de almacenamiento centralizado y procesamiento distribuido, con el objetivo de trabajar con mayores volúmenes de información y permitir la administración de los modelos mediante una aplicación web.

Como el primer paso, la aplicación alojada en Amazon EC2 obtendría los archivos originales proporcionados mediante Google Drive y los almacenaría en el bucket /raw de Amazon S3. Posteriormente, se iniciarían los trabajos de procesamiento mediante Amazon EMR, donde Spark accedería a los archivos almacenados y ejecutaría las tareas de limpieza, transformación y preparación de los datos de manera distribuida entre distintos nodos.

Una vez preparados los datos, se realizaría el entrenamiento del modelo utilizando Spark para cómputo distribuido de nuevo. Al finalizar este proceso, los modelos generados y sus resultados se almacenarían en el bucket /models de Amazon S3. Al mismo tiempo, se registraría en Amazon RDS la información correspondiente a cada modelo, incluyendo su nombre, versión, fecha y hora de creación, así como la ubicación del archivo dentro de S3.

Finalmente, la aplicación web podría consultar los registros en RDS para identificar los modelos disponibles y acceder a sus archivos en S3. Esto permitiría cargar los modelos entrenados para generar predicciones a partir de nuevos datos ingresados por los usuarios.

Este flujo mantendría las mismas etapas generales de preparación, modelado y evaluación utilizadas en el entorno de desarrollo, pero cambiaría la infraestructura encargada de ejecutarlas. De esta manera, Amazon S3 se utilizaría para el almacenamiento, Amazon EMR y Spark para el procesamiento distribuido, Amazon RDS para administrar los metadatos y Amazon EC2 para ejecutar la aplicación web y coordinar las operaciones del sistema.

En conjunto, ambos escenarios permiten realizar el procesamiento y análisis de los datos proporcionados por el CAETEC. La diferencia se encuentra en la infraestructura utilizada, ya que el entorno de desarrollo aprovecha los recursos de Google, mientras que el entorno de producción propone servicios de AWS para facilitar el procesamiento de mayores volúmenes de información y el acceso a los modelos mediante una aplicación web desplegada.

4. Carga, Procesamiento y Modelado de Datos

Con el objetivo de comprobar la viabilidad de construir un flujo de datos similar al descrito y la capacidad de usar Spark para cómputo distribuido, se desarrolló una prueba de concepto mediante PySpark en Google Colab. Esta implementación busca verificar que es posible construir un flujo de datos que incluya extracción, limpieza, almacenamiento, entrenamiento y evaluación de un modelo, utilizando los datos de ordeño proporcionados por el CAETEC.

Para desarrollar esta prueba de concepto se implementaron las siguientes etapas:

1. **Extracción de datos:** Se cargaron los archivos originales desde Google Drive utilizando Pandas para su lectura inicial y PySpark para su posterior procesamiento.
2. **Limpieza y preparación:** Se eliminaron columnas con información insuficiente, registros duplicados y valores inconsistentes. Además, se transformaron variables de tiempo y se prepararon todos los datos necesarios para identificar posibles colas.
3. **Almacenamiento:** Los datos procesados se guardaron en formato Parquet dentro de Google Drive. Posteriormente, se realizó una lectura de verificación para comprobar que se conservaran los registros y columnas originales del conjunto procesado.
4. **Preparación para el modelo:** Se definió una variable objetivo binaria que representa la posible presencia de colas y se seleccionaron variables numéricas relacionadas con los registros de ordeño. Estas se agruparon usando VectorAssembler de PySpark ML.
5. **Entrenamiento:** Se implementó un modelo de regresión logística, dividiendo los datos en 80% para entrenamiento y 20% para prueba. Se utilizó ParamGridBuilder para comparar configuraciones, junto con validación cruzada de 3 particiones.
6. **Evaluación:** Se utilizó la métrica ROC-AUC para evaluar el desempeño del modelo durante la validación cruzada y después sobre el conjunto de prueba independiente.

La implementación permitió construir un pipeline de procesamiento y modelado utilizando PySpark, integrando la lectura de archivos, las transformaciones de datos, el almacenamiento en formato Parquet y herramientas de aprendizaje automático, en este caso, de Spark MLlib.

5. Reflexión sobre el Uso de Big-Data

Para determinar si el proyecto requiere el uso de tecnologías de Big Data, se realizó un análisis mediante el enfoque de las 5 V's, considerando las características del conjunto de datos proporcionado por el CAETEC para nuestro proyecto, compuesto por 404 archivos estructurados en 44 columnas que representan aproximadamente 237.7 MB en memoria.

1. **Volumen:** El conjunto de datos tiene un tamaño aproximado de 237.7 MB en memoria, lo que es un volumen pequeño que puede procesarse utilizando la RAM de una computadora común o los recursos disponibles en Google Colab. No se considera necesario utilizar herramientas distribuidas como Spark para el desarrollo actual.
2. **Velocidad:** Los datos se proporcionan mediante archivos históricos y no existe la necesidad de procesar información en tiempo real ni de realizar una ingesta continua de grandes cantidades de registros. Por lo tanto, el procesamiento por lotes local resulta suficiente para la preparación, el análisis y el entrenamiento del modelo.
3. **Variedad:** La información se encuentra organizada en archivos con datos estructurados. Esto facilita su integración, limpieza y transformación mediante Pandas y NumPy, sin requerir tecnologías para manejar distintos formatos de información.
4. **Veracidad:** Los registros provienen directamente del robot de ordeño del CAETEC, una fuente de información confiable. Sin embargo, es necesario realizar validación y limpieza para detectar valores que puedan afectar la calidad y resultados del modelo.
5. **Valor:** El análisis de los datos permite identificar patrones en la formación de colas en el sistema de ordeño. Los resultados podrían aportar información útil sobre este comportamiento y apoyar la toma de decisiones para mejorar la eficiencia del proceso.

A partir del análisis de las 5 V's, decidimos que el proyecto no requiere un enfoque orientado a Big Data, debido principalmente al volumen reducido de información, su estructura definida y la ausencia de procesamiento en tiempo real. Por lo tanto, las herramientas de desarrollo seleccionadas son suficientes, aunque podría adoptarse la arquitectura de producción propuesta si aumentaran las necesidades de procesamiento y almacenamiento.